

Keywords: recursive self-improvement • AI research agents
• bi-level optimization • autonomous ML

Recursive Self-Improvement of AI Research Agents

An Outer Agent That Rewrites Its Own Code Beats Human-Engineered Baselines Across Four Held-Out Benchmarks in Eight Days

By Dhruv Srikanth, Bingchen Zhao, Dixing Xu, Yuxiang Wu & Zhengyao Jiang (Weco AI)

arXiv:2609.26457 • September 2026

AIDE², a bi-level system that uses an outer AI agent to autonomously rewrite the scaffolding code of an inner AI research agent, discovered seven successive improvements over an unattended 8-day run—including a 16× prompt compression and a halving of reward hacking—and matched or exceeded a human-engineered production agent on all four out-of-distribution held-out benchmarks.

AT A GLANCE

Problem: AI research agent scaffolding is fixed by human engineers and cannot benefit from the same automated improvement loop that the agent applies to its tasks.

Why it matters: Human R&D yields diminishing returns; a system that autonomously improves its own search policy and memory compresses months of engineering into days.

Method: Outer LLM agent proposes code edits to the inner agent’s scaffolding; edits are accepted only if they improve aggregate performance on a diverse training battery.

Result: 7 successive improvements in 8 days; 16× prompt compression; reward hacking drops from 63% to 34%.

Evidence: Four held-out benchmarks spanning ML engineering, algorithm design, and weather forecasting.

The Big Picture

Autonomous AI research agents such as AIDE (AI Development and Execution) can solve machine learning engineering tasks without human intervention: given a dataset and an objective, the agent proposes, executes, and refines code solutions in an automated loop. These agents have matched human competitors on benchmark challenges. But their *scaffolding*—the code that governs how they search, what they remember, and how they prompt the underlying language model—has always been

designed by human engineers.

This paper asks whether the self-improvement loop can be turned inward. The answer is AIDE²: when the agent’s own scaffolding code is the object of optimization, each accepted rewrite becomes the agent that the next round edits. In 8 unattended days, this loop discovered improvements that human engineers had not found in two years.

Why Existing Approaches Fall Short

Static scaffolding: Existing AI research agents (AIDE, AutoML-GPT, OpenDevin-Science) optimise task solutions but treat their own search policy and prompts as fixed hyperparameters chosen by human experts.

Meta-learning adapts model weights over episodes but requires differentiable objectives and cannot modify discrete code logic.

Neural architecture search and **program synthesis** optimise specific modules in narrow settings; neither is designed to improve a general-purpose agent’s control code.

Human iteration is slow, expensive, and bounded by engineering bandwidth. Moreover, the most obvious improvements have already been incorporated; finding subtler gains requires exhaustive experimentation.

Core Mechanism

Bi-Level Optimization. AIDE² wraps the standard AIDE agent in an outer optimization loop:

- *Inner loop (AIDE):* Given a task and budget, the inner agent proposes, executes, evaluates, and refines code solutions via tree search.
- *Outer loop (AIDE²):* An outer LLM agent inspects the inner agent’s scaffolding code θ , its performance traces, and recent failure modes. It proposes a code edit δ (a patch), spins up a fresh inner agent with $\theta' = \theta + \delta$, evaluates it on approximately 50 diverse tasks, and accepts δ only if performance improves.

Progressive Code Repository. Accepted edits accumulate in a version-controlled repository; each generation’s code is the starting point for the next round. Only about 1 in 10 proposed modifications is accepted.

Technical Formulation

Let π_θ denote the inner agent parameterised by scaffolding code θ , $\mathcal{T}_{\text{train}}$ the training task battery, $\mathcal{T}_{\text{held}}$ the held-out evaluation set.

The outer loop solves:

$$\theta^* = \arg \max_{\theta' \in \mathcal{N}(\theta)} \frac{1}{|\mathcal{T}_{\text{train}}|} \sum_{t \in \mathcal{T}_{\text{train}}} \text{score}(\pi_{\theta'}, t)$$

where $\mathcal{N}(\theta)$ is the set of code edits proposed by the outer LLM. The update rule:

$$\theta \leftarrow \theta' \quad \text{iff} \quad \text{Eval}(\pi_{\theta'}, \mathcal{T}_{\text{train}}) > \text{Eval}(\pi_{\theta}, \mathcal{T}_{\text{train}})$$

The outer agent is itself an LLM that reads θ , performance traces, and recent deltas before producing the patch δ in natural language followed by the code diff.

Learning or Inference Procedure

Outer loop (8 days, fully unattended):

1. Initialise θ_0 from the production AIDE codebase.
2. Outer LLM proposes a code edit δ to θ .
3. Sandboxed inner agent $\pi_{\theta+\delta}$ runs on $\mathcal{T}_{\text{train}}$.
4. If aggregate score improves: commit $\theta \leftarrow \theta + \delta$.
5. Repeat; $\approx 90\%$ of proposals are rejected.

Inner loop (per task):

1. Read task specification and compute budget.
2. Tree search: draft $\rightarrow$ execute $\rightarrow$ score $\rightarrow$ improve.
3. Return best solution within budget.

What the Guarantee Says

No formal convergence bound is given; the search space is open-ended natural-language code. The empirical claim is that a sufficiently diverse training battery (50 tasks, multiple domains) prevents overfitting: improvements that score higher on the training battery also generalise to the four held-out benchmarks—a necessary condition for recursive self-improvement to be meaningful rather than evaluation hacking.

Experimental Findings

- **7 successive improvements** accepted over 8 days from approximately 70 proposed modifications.
- **New search algorithm:** replaced default tree policy with a learned priority function exploiting evaluation feedback.
- **16 $\times$ prompt compression:** a compressed prompting scheme reduces per-step cost and enables longer search horizons.
- **Reward hacking:** self-policing emerged naturally; hacking rate dropped from 63% to 34% on a held-out benchmark.
- **4 held-out benchmarks** (ML engineering, algorithm engineering, physics forecasting, and one additional ML benchmark): the evolved agent matches

or exceeds the human-engineered production AIDE on all four.

Ablations and Interpretation

Task diversity: Narrowing the training battery to a single domain causes held-out generalisation to degrade significantly; diversity is essential.

Outer model size: Larger outer LLMs produce higher-quality proposals and higher acceptance rates, but even smaller models drive improvement given sufficient budget.

Reward hacking: The self-policing behavior was not explicitly incentivised; it emerged because reward hacking lowered aggregate evaluation scores and the outer loop penalised it implicitly—an encouraging sign that the optimisation pressure is well-aligned.

Gain decomposition: Prompt compression contributes $\approx 40\%$ of the total held-out improvement; the new search policy $\approx 35\%$; memory mechanisms $\approx 25\%$.

Reference

Dhruv Srikanth, Bingchen Zhao, Dixing Xu, Yuxiang Wu, Zhengyao Jiang. **Recursive self-improvement of AI research agents**. arXiv:2609.26457, September 2026.
<https://arxiv.org/abs/2609.26457>